

Diseño de Patrones



Universidad
Tecnológica
del Perú

Antipatrones de Diseño de Software

Prácticas o decisiones que parecen solucionar problemas, pero generan dificultades de mantenimiento, escalabilidad y evolución del software.

1. Clase Dios (God Class)

Una clase concentra demasiadas responsabilidades.

Problemas:

- Difícil mantenimiento
- Difícil prueba
- Viola responsabilidad única (SRP)



- Ejemplo: una clase que registra clientes, pagos, reportes y correos.

```
1  class SistemaBanco {
2
3      void registrarCliente(){}
4
5      void realizarDeposito(){}
6
7      void calcularIntereses(){}
8
9      void generarReporte(){}
10
11     void enviarCorreo(){}
12
13     void validarUsuario(){}
14 }
```

Problema:

- La clase crece demasiado.
- Es difícil modificarla sin romper otras funciones.
- No cumple el principio de responsabilidad única (SRP).

Solución:

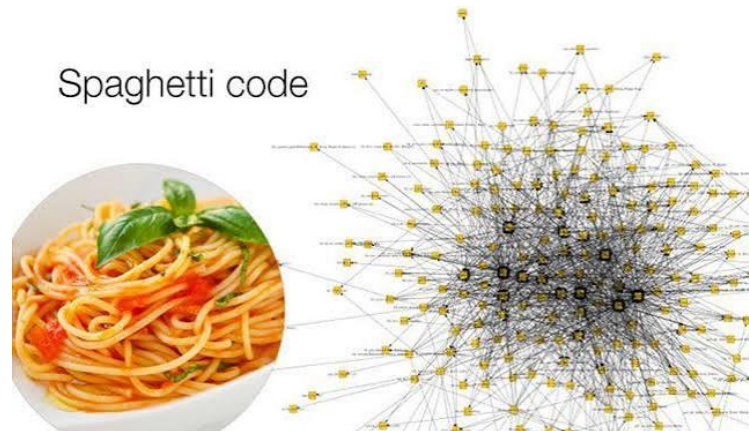
Separar responsabilidades:

2. Código Espagueti

Código desordenado con muchos if, ciclos y lógica mezclada.

Problemas:

- Difícil lectura
- Difícil modificación
- Mayor riesgo de errores



```
1  if(usuario!=null){
2
3      if(usuario.getTipo()=="Admin"){
4
5          if(usuario.getEstado()=="Activo"){
6
7              if(usuario.getPermiso()==true){
8
9                  abrirSistema();
10
11              }
12
13          }
14
15      }
16
17 }
```

Problema:

- Difícil de leer.
- Difícil de probar.
- Cualquier cambio puede generar errores.

Solución:

Crear métodos independientes

```
1  if(usuarioPuedeIngresar()){
2      abrirSistema();
3  }
```

3. Código Duplicado (Copy-Paste Programming)

Copiar y pegar el mismo código en diferentes lugares.

- Problema:
- Un cambio debe hacerse en múltiples partes del sistema.
- Solución: reutilizar funciones o componentes.

```
1 // Registro cliente
2 System.out.println("Validando DNI");
3 System.out.println("Guardando cliente");
4
5
6 // Registro empleado
7 System.out.println("Validando DNI");
8 System.out.println("Guardando empleado");
```

Problema:

Si cambia la validación del DNI, hay que modificar muchos lugares.

Solución:

Crear una función:

```
1 void validarDNI(){
2     System.out.println("Validando DNI");
3 }
```


4. Martillo de Oro (Golden Hammer)

Usar siempre la misma tecnología o solución para todos los problemas.

- Ejemplo:
- Usar una base de datos relacional para cualquier tipo de aplicación sin analizar necesidades.

Ejemplo:

Un programador usa bases de datos relacionales para todo:

- Sistema bancario
- Chat en tiempo real
- Sistema de recomendaciones

Problema:

No analiza si la herramienta es adecuada

Solución:

Elegir la tecnología según la necesidad.



5. Objeto Dios (God Object)

Un objeto controla demasiadas funciones del sistema.

- Problema:
- Rompe la separación de responsabilidades y aumenta el acoplamiento.

```
1  class Usuario {
2
3      String nombre;
4
5      void pagarFactura(){}
6
7      void enviarCorreo(){}
8
9      void generarReporte(){}
10
11     void conectarBD(){}
12
13 }
```

Problema:

El usuario termina haciendo funciones que no corresponden.

Solución:

Separar:

Usuario

|

|

PagoService

CorreoService

ReporteService

6. Parche sobre Parche

Agregar soluciones temporales una encima de otra sin corregir el problema original.

- Resultado:
- Código complejo y difícil de mantener.

7. Big Ball of Mud

Sistema sin arquitectura clara que crece con código mezclado, documentación pobre y módulos desordenados.

Ejemplo:

Proyecto después de varios años:

Proyecto

- |
- | -- archivos viejos
- | -- código repetido
- | -- módulos mezclados
- | -- funciones sin usar
- | -- documentación inexistente

Problema:

Nadie entiende completamente el sistema.

Resumen

Los antipatrones generan:

- Mayor costo de mantenimiento
 - Código difícil de entender
 - Problemas de escalabilidad
 - Más errores
-
- La solución es aplicar buenas prácticas, principios SOLID y patrones adecuados.